

UNIVERSIDADE DO ESTADO DO AMAZONAS – UEA
ESCOLA SUPERIOR DE TECNOLOGIA

ENGENHARIA DA COMPUTAÇÃO

**RELATÓRIO DE EXPERIMENTOS
OPENGL/GLUT E OPENCV**

Beatriz Guedes da Silva

Manaus – AM
2026

Sumário

1	Objetivos	3
1.1	Objetivo geral	3
1.2	Objetivos específicos	3
2	Introdução Teórica	3
2.1	OpenGL	3
2.2	GLUT e FreeGLUT	3
2.3	OpenCV	4
2.4	Representação de cores	4
2.4.1	Modelo RGB	4
2.4.2	Modelo BGR	4
2.4.3	Modelo HSV	4
2.5	Canal Alpha (Modelo RGBA)	4
2.6	Profundidade de Cor	5
3	Descrição do Ambiente de Desenvolvimento	5
3.1	Compilador	5
3.2	OpenGL e FreeGLUT	5
4	Metodologia e Desenvolvimento	5
4.1	Configuração do ambiente	5
4.2	Desenvolvimento das aplicações	6
4.3	Procedimentos experimentais	6
5	Experimentos com OpenGL/GLUT	6
5.1	Script 1	6
5.1.1	Objetivo	6
5.1.2	Metodologia	7
5.1.3	Implementação	7
5.1.4	Resultado	8
5.2	Script 3	8
5.2.1	Objetivo	8
5.2.2	Metodologia	9
5.2.3	Implementação	9
5.2.4	Resultado	10
6	Experimentos com OpenCV	10
6.1	Script 2	10

6.1.1	Objetivo	10
6.1.2	Metodologia	11
6.1.3	Implementação	11
6.1.4	Resultado	12
7	Resultados Obtidos	13
7.1	Resultados do OpenGL/FreeGLUT	13
7.2	Resultados do OpenCV	13
8	Análise e Discussão dos Resultados	14
9	Comparação dos Padrões e Espaços de Representação de Cores	14
10	Conclusão	15
A	Script 1	18
B	Script 2	19
C	Script 3	22

1 Objetivos

1.1 Objetivo geral

Introduzir os conceitos de representação e manipulação de cores em Computação Gráfica e Processamento Digital de Imagens, explorando a utilização de OpenGL/GLUT (ou FreeGLUT) para criação e renderização de objetos gráficos e do OpenCV para leitura, manipulação e processamento de imagens.

1.2 Objetivos específicos

- Configurar um ambiente de desenvolvimento contendo OpenGL, GLUT/FreeGLUT e OpenCV;
- Desenvolver aplicações básicas utilizando OpenGL;
- Desenvolver aplicações para processamento e manipulação de imagens utilizando OpenCV;
- Investigar diferentes padrões e espaços de representação de cores;
- Comparar os resultados obtidos nos experimentos;
- Documentar o processo de desenvolvimento e os resultados obtidos.

2 Introdução Teórica

2.1 OpenGL

OpenGL (*Open Graphics Library*) é uma API destinada à renderização de gráficos bidimensionais e tridimensionais.

A biblioteca fornece uma interface para comunicação com recursos gráficos, permitindo a definição de primitivas, transformações, cores, iluminação e outros elementos relacionados à renderização.

2.2 GLUT e FreeGLUT

A GLUT (*OpenGL Utility Toolkit*) fornece funcionalidades auxiliares para aplicações OpenGL, especialmente para criação de janelas, gerenciamento de eventos e interação com o usuário.

O FreeGLUT é uma implementação de código aberto e moderna da especificação original da GLUT.

2.3 OpenCV

OpenCV (*Open Source Computer Vision Library*) é uma biblioteca voltada para visão computacional e processamento de imagens.

Entre suas funcionalidades encontram-se operações de leitura, escrita, transformação e análise de imagens, além de ferramentas para detecção de características e processamento de vídeo.

2.4 Representação de cores

A representação digital de cores pode ser realizada por diferentes modelos e espaços de cores. Entre os mais conhecidos encontram-se

2.4.1 Modelo RGB

No modelo RGB, uma cor é representada pela combinação das componentes:

$$C = (R, G, B) \quad (1)$$

Em imagens de 8 bits por canal, cada componente normalmente assume valores no intervalo de 0 a 255.

2.4.2 Modelo BGR

BGR não é um espaço de cores propriamente dito, mas sim uma representação de uma imagem onde a ordem dos canais de cor é diferente da representação RGB tradicional. Os valores de cor dos pixels na imagem permanecem os mesmos, mas a ordem dos canais de cor é invertida, com o azul sendo o primeiro canal, o verde o segundo e o vermelho o terceiro.

2.4.3 Modelo HSV

O modelo HSV representa as cores por meio de três componentes: matiz (*Hue*), saturação (*Saturation*) e valor (*Value*).

Esse modelo pode ser particularmente útil em tarefas de segmentação e identificação de regiões de uma imagem com base em características cromáticas.

2.5 Canal Alpha (Modelo RGBA)

O canal alpha atua como uma quarta componente associada aos canais vermelho, verde e azul (RGB), formando a representação RGBA. Em imagens de 8 bits, o valor do alpha normalmente varia de 0 a 255, em que 0 representa transparência total e

255 representa opacidade total. Valores intermediários representam diferentes níveis de transparência.

2.6 Profundidade de Cor

A profundidade de cor refere-se ao número de cores distintas que podem ser exibidas ou representadas em uma imagem ou em uma tela. Determina a faixa e a variedade de cores que podem ser reproduzidas. Quanto maior a profundidade da cor, mais cores podem ser exibidas, resultando em imagens mais realistas e vibrantes.

3 Descrição do Ambiente de Desenvolvimento

3.1 Compilador

Foi utilizado o compilador GCC (GNU Compiler Collection) para a compilação das aplicações desenvolvidas. O desenvolvimento foi realizado utilizando a IDE CLion, que fornece integração com o sistema de build CMake e foi configurada para utilizar o GCC como compilador.

3.2 OpenGL e FreeGLUT

A configuração do ambiente incluiu a instalação das bibliotecas necessárias para o desenvolvimento de aplicações utilizando OpenGL e FreeGLUT. O ambiente disponibiliza a especificação OpenGL na versão 4.6, com implementação fornecida pelo Mesa 26.1.8, e o pacote FreeGLUT na versão 3.8.0.

Para o desenvolvimento da aplicação, foi utilizada a biblioteca OpenCV na versão 4.13.0, instalada no sistema e localizada pelo CMake do IDE.

4 Metodologia e Desenvolvimento

O desenvolvimento foi dividido em etapas, iniciando pela configuração do ambiente e posteriormente pela implementação dos experimentos utilizando OpenGL/GLUT e OpenCV.

4.1 Configuração do ambiente

Para a configuração do ambiente, foi criado um projeto utilizando a IDE CLion, com o objetivo de desenvolver três códigos independentes, cada um destinado à resolução de uma das questões propostas pelo professor. Para organizar os arquivos, foram criadas

três pastas, uma para cada código, além de uma pasta destinada ao armazenamento das imagens utilizadas durante o desenvolvimento do trabalho.

Posteriormente, foram configurados quatro arquivos `CMakeLists.txt`: um na pasta principal do projeto e um em cada uma das pastas correspondentes aos códigos. Essa organização permitiu estruturar individualmente cada aplicação, facilitando seu gerenciamento, configuração e processo de compilação por meio do CMake.

4.2 Desenvolvimento das aplicações

Foi utilizada a linguagem C++ para o desenvolvimento das aplicações. Foram desenvolvidos três scripts independentes: os Scripts 1 e 3 foram destinados à demonstração da utilização das bibliotecas OpenGL e Glut_FreeGLUT, enquanto o Script 2 foi desenvolvido para demonstrar a utilização da biblioteca OpenCV.

4.3 Procedimentos experimentais

Para cada experimento foram registrados:

1. objetivo do experimento;
2. metodologia utilizada;
3. código desenvolvido;
4. resultado obtido;
5. captura de tela da execução;
6. análise do resultado.

5 Experimentos com OpenGL/GLUT

5.1 Script 1

5.1.1 Objetivo

Desenvolver uma aplicação utilizando GLUT/FreeGLUT e OpenGL que apresente diferentes objetos geométricos, explorando:

- definição de cores dos objetos;
- combinação das componentes RGB;
- diferentes intensidades de vermelho, verde e azul;

- transparência;
- sobreposição de objetos;
- efeitos da ordem de renderização sobre objetos transparentes.

5.1.2 Metodologia

Para o desenvolvimento do Script 1, foi definida uma cena com fundo preto contendo três objetos geométricos. O primeiro objeto consiste em um quadrado vermelho sólido, posicionado mais ao fundo e à esquerda da cena. Em seguida, foi inserido um triângulo verde, localizado na região central e com um nível moderado de transparência. Por fim, foi adicionado um quadrado azul, posicionado mais à direita e à frente dos demais objetos, apresentando um nível de transparência maior.

A disposição dos objetos foi definida de forma a demonstrar o uso de transparência e a sobreposição de elementos utilizando OpenGL e GLUT_FreeGLUT.

5.1.3 Implementação

O código desenvolvido é composto por uma função auxiliar, responsável pela exibição da cena, e pela função `main`, responsável pela configuração inicial da aplicação.

A função `display` é responsável por desenhar os elementos da cena. Inicialmente, a tela é limpa e, em seguida, é habilitado o uso de transparência por meio do recurso de *blending* do OpenGL. Posteriormente, os objetos são desenhados individualmente, seguindo a ordem definida na metodologia.

O primeiro objeto desenhado é o quadrado vermelho, definido pela componente de cor RGB (1,0,0) e com valor de alpha igual a 1,0, representando uma cor vermelha totalmente opaca. O triângulo verde é definido pelas componentes (0,1,0) e possui alpha igual a 0,6, apresentando transparência parcial. Por fim, o quadrado azul é definido pelas componentes (0,0,1) e possui alpha igual a 0,2, resultando em um objeto mais transparente.

Os valores das componentes de cor utilizados pelo OpenGL são normalizados no intervalo de 0 a 1. Dessa forma, os valores 1 e 0 representam, respectivamente, a intensidade máxima e mínima de cada componente de cor. A componente alpha também utiliza esse intervalo, sendo 1,0 correspondente à opacidade total e valores menores correspondentes a diferentes níveis de transparência.

A função `main` é responsável pela configuração inicial da aplicação. Nela, é definido o modo de exibição utilizando RGBA e *double buffering*, além do tamanho da janela e do título apresentado ao usuário. Por fim, é registrada a função `display` como responsável pelo desenho da cena e o loop principal do FreeGLUT é iniciado, permitindo a exibição e atualização da janela.

5.1.4 Resultado

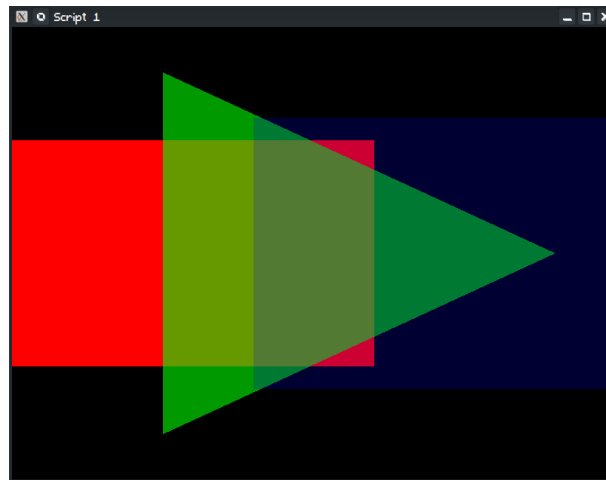


Figura 1: Resultado do Primeiro Experimento.

A imagem demonstra o funcionamento esperado da aplicação, apresentando os três objetos geométricos sobre um fundo preto. É possível observar o quadrado vermelho opaco, o triângulo verde parcialmente transparente e o quadrado azul com maior nível de transparência. Nas regiões em que os objetos se sobrepõem, ocorre a composição das cores devido aos diferentes níveis de transparência aplicados.

As cores resultantes nas regiões de interseção podem ser explicadas pela composição dos valores RGB dos objetos envolvidos, juntamente com seus respectivos valores de alpha. Dessa forma, tanto os níveis de transparência quanto a ordem em que os objetos são desenhados influenciam diretamente no resultado visual da cena.

Esse comportamento será explorado novamente no Script 3, permitindo observar de forma mais evidente a influência da ordem de desenho sobre o resultado da composição dos objetos.

5.2 Script 3

5.2.1 Objetivo

Explorar a utilização de transparência (Alpha Blending) em OpenGL, criando pelo menos dois objetos parcialmente transparentes e analisando:

- efeito de diferentes valores do canal Alpha;
- sobreposição de objetos;
- ordem de desenho;
- configuração do blending no OpenGL.

5.2.2 Metodologia

Para este script, foram criadas duas cenas semelhantes, porém com uma diferença fundamental: a ordem de desenho dos objetos. Em ambas as cenas, foram utilizados dois objetos geométricos de formatos e níveis de transparência distintos, posicionados de forma a se sobreporem parcialmente.

O primeiro objeto é de cor vermelha, enquanto o segundo é de cor azul. Na primeira cena, os objetos são desenhados seguindo uma determinada ordem e, na segunda, essa ordem é invertida. Dessa forma, é possível comparar os resultados e demonstrar a influência da ordem de desenho na composição dos objetos transparentes.

5.2.3 Implementação

Este script segue uma estrutura de funções semelhante à utilizada no Script 1. A função `display` é responsável por desenhar duas cenas distintas na mesma janela, sendo uma localizada na parte superior e outra na parte inferior. Ambas as cenas possuem os mesmos objetos, porém com a ordem de desenho invertida.

Na cena superior, o objeto vermelho é desenhado primeiro, utilizando as componentes RGB (1,0,0) e alpha igual a 0,7. Em seguida, o objeto azul é desenhado sobre o vermelho, permitindo observar o resultado da composição das cores com diferentes níveis de transparência.

Na cena inferior, a ordem é invertida: o objeto azul, definido pelas componentes RGB (0,0,1) e alpha igual a 0,4, é desenhado primeiro, seguido pelo objeto vermelho. Essa alteração permite comparar visualmente os efeitos produzidos pela mudança na ordem de desenho dos objetos.

A função `main` apresenta uma configuração semelhante à do Script 1, incluindo a definição do modo de exibição com RGBA e *double buffering*, o dimensionamento da janela, a definição de seu título e a configuração necessária para sua exibição ao usuário.

5.2.4 Resultado

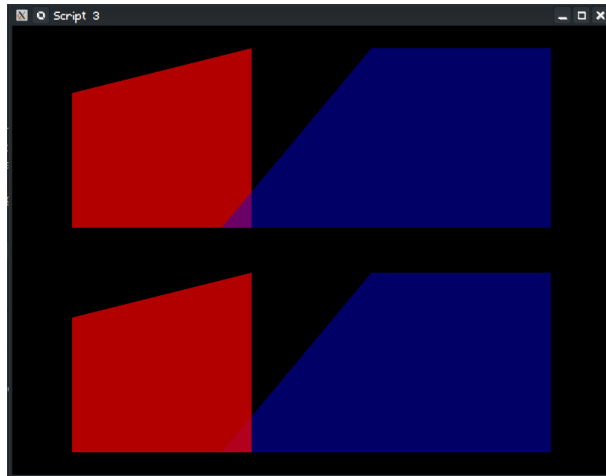


Figura 2: Resultado do Terceiro Experimento.

Pode-se observar que a ordem de desenho dos objetos altera diretamente o resultado visual da composição. Quando o objeto mais transparente é desenhado sobre o objeto mais opaco, este último permanece bastante evidente, inclusive na região de interseção entre os dois objetos. Por outro lado, quando o objeto mais opaco é desenhado posteriormente, sua maior opacidade reduz significativamente a visibilidade da região de interseção do objeto transparente, fazendo com que essa parte fique praticamente encoberta.

6 Experimentos com OpenCV

6.1 Script 2

6.1.1 Objetivo

Desenvolver, utilizando OpenCV, uma rotina capaz de:

- carregar uma imagem a partir de arquivo;
- acessar e manipular suas componentes de cor;
- gerar diferentes versões modificadas da imagem;
- converter a imagem entre diferentes espaços de cor, como BGR, RGB, HSV e escala de cinza;
- apresentar e comparar os resultados obtidos.

6.1.2 Metodologia

Para esse script, foi utilizada a seguinte imagem como base:



Figura 3: Foto utilizada para o experimento 2.

A imagem foi submetida a diferentes manipulações, com ênfase na alteração de suas cores e componentes de cor. Foram realizadas transformações para a obtenção de um efeito sépia, a adição de granulação e o realce da tonalidade roxa na imagem. Além disso, a imagem foi convertida para diferentes modelos e representações de cor, incluindo RGB, escala de cinza (Gray), HSV e a conversão entre os canais RGB e BGR.

6.1.3 Implementação

A partir da imagem carregada pela função `CarregarImagem`, são realizadas as diferentes manipulações propostas. A função `ComponentesCor` concentra as operações de processamento da imagem, permitindo alterar suas características de cor e aparência sem modificar suas dimensões.

Para a obtenção do efeito sépia, foi aplicada uma transformação utilizando uma matriz (*kernel*) específica, responsável por modificar as componentes de cor da imagem e produzir a tonalidade característica do efeito sépia. Nesse caso, a função `cv::transform` realiza uma transformação linear sobre os canais da imagem, utilizando os valores definidos na matriz de transformação.

Para a aplicação do efeito de granulação, foi adicionado um ruído à imagem em intervalos de 30 pixels. Dessa forma, a redução desse intervalo aumenta a quantidade de pontos afetados e, consequentemente, intensifica o efeito de granulação.

Por fim, para realçar a tonalidade roxa da imagem, foi criado um mapa de valores destinado a evidenciar essa componente de cor. Esse mapa foi posteriormente combinado à imagem original por meio da função `MixChannels`, permitindo modificar a composição dos canais de cor e produzir o efeito visual desejado.

6.1.4 Resultado

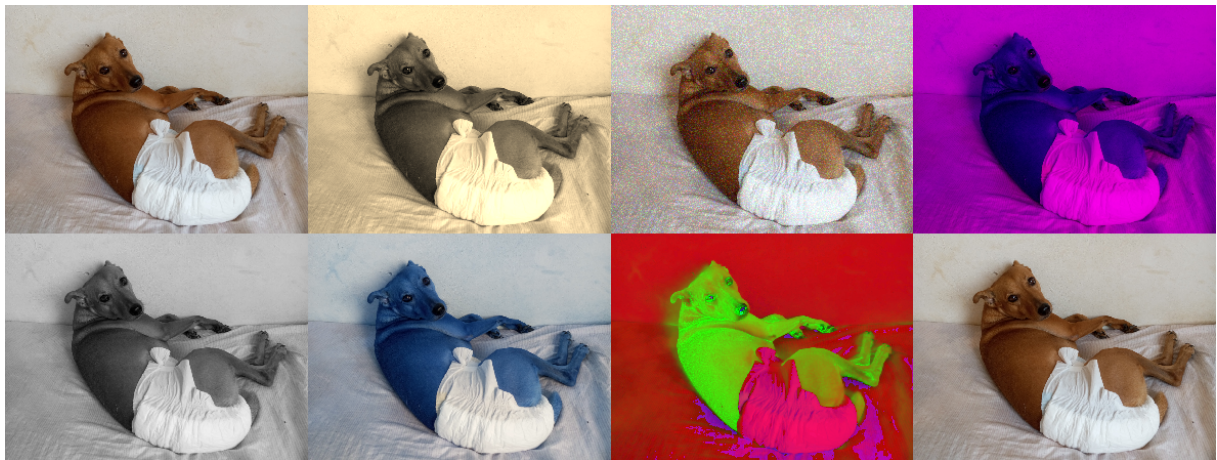


Figura 4: Grade de Fotos com o Resultado do Experimento 2.

A Figura 4 apresenta os resultados das diferentes operações aplicadas à imagem, organizados em oito versões:

1. imagem original;
2. imagem com efeito sépia;
3. imagem com efeito de granulação;
4. imagem com realce da tonalidade roxa;
5. imagem convertida para escala de cinza;
6. imagem representada no espaço de cor RGB;
7. imagem representada no espaço de cor HSV;
8. imagem representada na ordem de canais BGR, após a conversão da imagem para RGB.

Os resultados demonstram que diferentes técnicas de processamento, como transformações lineares, manipulação e combinação de canais e adição de ruído, podem modificar significativamente a aparência de uma imagem e suas características de cor.

Além disso, a conversão entre diferentes representações de cor é um recurso importante no processamento de imagens. No OpenCV, as imagens são carregadas por padrão na ordem de canais BGR, diferentemente da representação RGB utilizada em diversas aplicações e ferramentas. Dessa forma, a compreensão e a conversão entre essas representações são importantes para garantir a correta interpretação das cores durante o processamento.

7 Resultados Obtidos

Os experimentos realizados permitiram verificar, na prática, o funcionamento das bibliotecas utilizadas.

7.1 Resultados do OpenGL/FreeGLUT

Os experimentos realizados demonstraram o funcionamento das bibliotecas OpenGL e FreeGLUT na criação de cenas bidimensionais, bem como a utilização de cores e transparência na composição dos objetos.

Durante os experimentos, observou-se que os valores das componentes RGB utilizados pelo OpenGL são normalizados no intervalo de 0 a 1, diferentemente da representação de cores frequentemente utilizada em aplicações gráficas, na qual cada componente é representada no intervalo de 0 a 255. Apesar dessa diferença, a utilização de valores normalizados torna-se conveniente quando combinada à componente alpha, que também utiliza valores entre 0 e 1 para representar diferentes níveis de opacidade.

Os resultados também demonstraram que a ordem de desenho dos objetos possui influência direta sobre a composição das cores em regiões de sobreposição, especialmente quando são utilizados diferentes níveis de transparência. Objetos completamente opacos desenhados posteriormente podem ocultar os objetos que foram desenhados anteriormente.

Além disso, foi possível observar que recursos como a mistura de cores (*blending*) e a transparência podem ser habilitados ou desabilitados durante a execução da aplicação, permitindo controlar diretamente o comportamento visual dos objetos por meio das configurações do código.

7.2 Resultados do OpenCV

Os experimentos realizados demonstraram a ampla variedade de ferramentas disponibilizadas pelo OpenCV para o processamento e a manipulação de imagens. Foi possível observar que diferentes representações de cor podem ser utilizadas e convertidas conforme a necessidade da aplicação, sendo importante considerar que o OpenCV utiliza, por padrão, a ordem de canais BGR ao carregar imagens.

As manipulações realizadas demonstraram a possibilidade de aplicar diferentes técnicas de processamento, como transformações lineares por meio de matrizes, utilização de mapas de valores, adição de ruído e outras operações algébricas disponibilizadas pela biblioteca. Essas ferramentas permitem modificar tanto as características de cor quanto outros aspectos visuais das imagens de maneira programática.

Em algumas operações, os valores utilizados para representar as componentes de cor podem seguir intervalos normalizados, enquanto outras operações trabalham com va-

lores definidos de acordo com o tipo de dado e a representação utilizada pela imagem. Dessa forma, compreender a representação dos canais e os intervalos de valores utilizados em cada operação é fundamental para obter os resultados esperados durante o processamento.

8 Análise e Discussão dos Resultados

Os resultados obtidos nos experimentos permitiram demonstrar de forma satisfatória os principais conceitos propostos no trabalho, abrangendo a criação e composição de objetos gráficos com OpenGL e FreeGLUT, bem como o processamento e a manipulação de imagens utilizando OpenCV.

O OpenGL, em conjunto com o FreeGLUT para a criação e visualização das janelas, demonstrou ser uma ferramenta adequada para a criação de cenas gráficas a partir de formas geométricas. Por meio da definição de coordenadas, cores, transparência e ordem de desenho, foi possível construir e manipular uma cena bidimensional. Esses conceitos também constituem uma base para a representação de elementos em ambientes tridimensionais.

O OpenCV, por sua vez, demonstrou sua capacidade de processamento e manipulação de imagens. As operações realizadas permitiram modificar características visuais das imagens por meio de transformações matemáticas, manipulação de canais de cor, aplicação de ruídos e conversões entre diferentes representações de cor.

Também foi observado que o desempenho das aplicações é influenciado pelas características do hardware utilizado. Durante o desenvolvimento inicial, foi utilizada uma máquina com menor capacidade de processamento, resultando em tempos de execução superiores em determinadas operações. Dessa forma, embora as ferramentas utilizadas não exijam necessariamente hardware de alto desempenho para aplicações simples, o aumento da complexidade das operações e do volume de dados processados pode tornar recursos computacionais mais robustos relevantes para a redução do tempo de processamento.

9 Comparação dos Padrões e Espaços de Representação de Cores

A representação de cores pode ser realizada por meio de diferentes modelos e espaços de cores, cada um apresentando características específicas e sendo mais adequado a determinadas aplicações de processamento e representação de imagens.

Tabela 1: Comparação entre diferentes espaços de cores.

Espaço	Componentes	Características
RGB	Vermelho, verde e azul	Utilizado na representação direta de cores, sendo amplamente empregado em dispositivos de exibição e processamento de imagens.
BGR	Azul, verde e vermelho	Utilizado pelo OpenCV como ordem padrão dos canais durante o carregamento de imagens.
HSV	Matiz, saturação e valor	Separa informações relacionadas à tonalidade, à intensidade da cor e ao brilho, facilitando determinadas operações de análise e segmentação de imagens.

A comparação apresentada na Tabela 1 demonstra que diferentes espaços de cores podem ser utilizados de acordo com os objetivos do processamento. O modelo RGB é adequado para a representação direta das cores, enquanto o BGR está relacionado à forma como o OpenCV organiza os canais das imagens carregadas. Já o espaço HSV permite representar separadamente características como matiz, saturação e valor, podendo facilitar determinadas operações de análise e processamento.

Dessa forma, não existe necessariamente um único espaço de cores adequado para todas as aplicações. A escolha depende do objetivo do processamento e da forma como as características da imagem precisam ser representadas ou analisadas.

10 Conclusão

Os experimentos realizados permitiram compreender aspectos fundamentais relacionados à Computação Gráfica e ao Processamento Digital de Imagens.

A configuração do ambiente possibilitou o desenvolvimento de aplicações utilizando OpenGL/GLUT e OpenCV, permitindo observar na prática os conceitos apresentados na introdução teórica.

Os experimentos com OpenGL demonstraram as regras de como gerar uma imagem com objetos geométricos coloridos, transparentes e que conseguem gerar novas cores com a intersecção de objetos enquanto os experimentos utilizando OpenCV possibilitaram

aprender a manipular vários elementos da imagem e alterar sua escala de cor.

Por fim, a comparação entre diferentes espaços de representação de cores permitiu observar que cada modelo possui características próprias e pode ser mais adequado dependendo da aplicação.

Referências

- [1] Khronos Group. *OpenGL – The Industry’s Foundation for High Performance Graphics*. Disponível em: <https://www.opengl.org/>. Acesso em: 7 set. 2026.
- [2] FreeGLUT. *The FreeGLUT Project*. Disponível em: <https://freeglut.sourceforge.net/>. Acesso em: 7 set. 2026.
- [3] OpenCV. *OpenCV Documentation*. Disponível em: <https://docs.opencv.org/>. Acesso em: 7 set. 2026.
- [4] GONZALEZ, Rafael C.; WOODS, Richard E. *Digital Image Processing*. Pearson.
- [5] SHREINER, Dave et al. *OpenGL Programming Guide*. Addison-Wesley.

A Script 1

```

1
2 #include <GL/freeglut.h>
3
4 // Função responsável por desenhar a cena
5 void display() {
6     // Limpa a tela
7     glClear(GL_COLOR_BUFFER_BIT);
8     glLoadIdentity();
9
10    // Ativa a mistura de cores para permitir transparência
11    glEnable(GL_BLEND);
12    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
13
14    // 1. Quadrado Vermelho (Fundo - Totalmente opaco)
15    glColor4f(1.0f, 0.0f, 0.0f, 1.0f);
16    glBegin(GL_QUADS);
17    glVertex2f(-1.0f, 0.5f);
18    glVertex2f(0.2f, 0.5f);
19    glVertex2f(0.2f, -0.5f);
20    glVertex2f(-1.0f, -0.5f);
21    glEnd();
22
23    // 2. Triângulo Verde (Meio - Parcialmente transparente)
24    glColor4f(0.0f, 1.0f, 0.0f, 0.6f);
25    glBegin(GL_TRIANGLES);
26    glVertex2f(-0.5f, 0.8f);
27    glVertex2f(-0.5f, -0.8f);
28    glVertex2f(0.8f, 0.0f);
29    glEnd();
30
31    // 3. Quadrado Azul (Frente - Mais transparente)
32    glColor4f(0.0f, 0.0f, 1.0f, 0.2f);
33    glBegin(GL_QUADS);
34    glVertex2f(-0.2f, 0.6f);
35    glVertex2f(1.0f, 0.6f);
36    glVertex2f(1.0f, -0.6f);
37    glVertex2f(-0.2f, -0.6f);
38    glEnd();
39

```

```

40 // Desativa a mistura de cores
41 glDisable(GL_BLEND);
42
43 // Troca os buffers de tela
44 glutSwapBuffers();
45 }
46
47 // Função principal do programa
48 int main(int argc, char** argv) {
49     // Inicializa o GLUT
50     glutInit(&argc, argv);
51
52     // Configura o modo de exibição (Cores RGBA e Double Buffer)
53     glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGBA);
54
55     // Define o tamanho inicial da janela
56     glutInitWindowSize(800, 600);
57
58     // Cria a janela com um título
59     glutCreateWindow("Script 1");
60
61     // Define as funções de callback (eventos) do GLUT
62     glutDisplayFunc(display);
63
64     // Entra no loop principal de eventos do GLUT
65     glutMainLoop();
66
67     return 0;
68 }

```

Listing 1: Código-fonte da aplicação OpenGL/GLUT.

B Script 2

```

1
2
3 #include <opencv2/opencv.hpp>
4 #include <iostream>
5 #include <string>
6 #include <vector>
7

```

```

8 cv::Mat CarregarImagem(std::string caminho) {
9     cv::Mat imagem = cv::imread(caminho);
10    if(imagem.empty()){
11        std::cout << "Foto nao encontrada" << std::endl;
12        return cv::Mat();
13    }
14    return imagem;
15 }
16
17 std::vector<cv::Mat> ComponentesCor(cv::Mat imagem) {
18
19     // S P I A
20     cv::Mat img1 = cv::Mat::zeros(imagem.size(), imagem.type());
21
22     cv::Mat kernelSepia = (cv::Mat_<float>(3, 3) <<
23     0.131f, 0.534f, 0.272f,
24     0.168f, 0.686f, 0.349f,
25     0.189f, 0.769f, 0.393f
26     );
27
28     cv::transform(imagem, img1, kernelSepia);
29
30
31     // GRANULADA
32     cv::Mat img2 = cv::Mat::zeros(imagem.size(), imagem.type());
33
34     cv::Mat ruido = cv::Mat::zeros(imagem.size(), imagem.type());
35
36     cv::randn(ruido,
37             cv::Scalar(0, 0, 0),
38             cv::Scalar(30, 30, 30));
39
40     cv::add(imagem, ruido, img2);
41
42     // MAIS ROXA
43
44     cv::Mat img3 = cv::Mat::zeros(imagem.size(), imagem.type());
45
46     int mapa3[] = {
47         2, 0,    // R -> B
48         0, 2    // B -> R

```

```

49     };
50
51     cv::mixChannels(&imagem, 1, &img3, 1, mapa3, 2);
52
53
54     return {img1, img2, img3};
55 }
56
57 std::vector<cv::Mat> ElementosCor(cv::Mat imagem) {
58     cv::Mat imagemcinza, imagemrgb, imagemhsv, imagembgr;
59
60     // BGR      escala de cinza
61     cv::cvtColor(imagem, imagemcinza, cv::COLOR_BGR2GRAY);
62
63     // BGR      RGB
64     cv::cvtColor(imagem, imagemrgb, cv::COLOR_BGR2RGB);
65
66     // BGR      HSV
67     cv::cvtColor(imagem, imagemhsv, cv::COLOR_BGR2HSV);
68
69     // RGB      BGR
70     cv::cvtColor(imagemrgb, imagembgr, cv::COLOR_RGB2BGR);
71
72     // Cinza (1 canal)      BGR (3 canais)
73     // Necessário para poder concatenar com as imagens de 3 canais
74     cv::cvtColor(imagemcinza, imagemcinza, cv::COLOR_GRAY2BGR);
75
76     return {imagemcinza, imagemrgb, imagemhsv, imagembgr};
77 }
78
79 int main(){
80     std::string caminhoimagem = "imagens/dogfofa.jpg"; // Substitua
81     pelo seu arquivo real
82     cv::Mat imagem = CarregarImagem(caminhoimagem);
83
84     // Se a imagem não carregou, encerra o programa para não dar
85     erro depois
86     if(imagem.empty()) return -1;
87
88     // Recebe e extrai as 3 imagens com canais trocados
89     std::vector<cv::Mat> comps = ComponentesCor(imagem);

```

```

88     cv::Mat imagem1 = comps[0];
89     cv::Mat imagem2 = comps[1];
90     cv::Mat imagem3 = comps[2];
91
92     // Recebe e extrai a imagem cinza e a RGB
93     std::vector<cv::Mat> elems = ElementosCor(imagem);
94     cv::Mat imagemc = elems[0];
95     cv::Mat imagemr = elems[1];
96     cv::Mat imagemh = elems[2];
97     cv::Mat imagemb = elems[3];
98
99     // Monta o painel
100    cv::Mat linha1, linha2, painelFinal;
101    cv::hconcat(std::vector<cv::Mat>{imagem, imagem1, imagem2,
102        imagem3}, linha1);
102    cv::hconcat(std::vector<cv::Mat>{imagemc, imagemr, imagemh,
103        imagemb}, linha2);
103    cv::vconcat(linha1, linha2, painelFinal);
104
105    // Exibe tudo de uma vez
106    cv::namedWindow("Script 2", cv::WINDOW_NORMAL);
107    cv::imshow("Script 2", painelFinal);
108    cv::waitKey(0);
109
110    return 0;
111 }

```

Listing 2: Código-fonte da aplicação OpenCV.

C Script 3

```

1
2 #include <GL/freeglut.h>
3
4 void display()
5 {
6     glClear(GL_COLOR_BUFFER_BIT);
7
8     // Ativa transparência
9     glEnable(GL_BLEND);
10    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);

```

```

11
12     // Vermelho primeiro, Azul depois
13     glColor4f(1.0f, 0.0f, 0.0f, 0.7f);
14     glBegin(GL_QUADS);
15     glVertex2f(-0.8f, 0.1f);
16     glVertex2f(-0.2f, 0.1f);
17     glVertex2f(-0.2f, 0.9f);
18     glVertex2f(-0.8f, 0.7f);
19     glEnd();
20
21     glColor4f(0.0f, 0.0f, 1.0f, 0.4f);
22     glBegin(GL_QUADS);
23     glVertex2f(-0.3f, 0.1f);
24     glVertex2f(0.8f, 0.1f);
25     glVertex2f(0.8f, 0.9f);
26     glVertex2f(0.2f, 0.9f);
27     glEnd();
28
29     // Azul primeiro, Vermelho depois
30     glColor4f(0.0f, 0.0f, 1.0f, 0.4f);
31     glBegin(GL_QUADS);
32     glVertex2f(-0.3f, -0.9f);
33     glVertex2f(0.8f, -0.9f);
34     glVertex2f(0.8f, -0.1f);
35     glVertex2f(0.2f, -0.1f);
36     glEnd();
37
38     glColor4f(1.0f, 0.0f, 0.0f, 0.7f);
39     glBegin(GL_QUADS);
40     glVertex2f(-0.8f, -0.9f);
41     glVertex2f(-0.2f, -0.9f);
42     glVertex2f(-0.2f, -0.1f);
43     glVertex2f(-0.8f, -0.3f);
44     glEnd();
45
46     glutSwapBuffers();
47 }
48
49 int main(int argc, char** argv)
50 {
51     glutInit(&argc, argv);

```



```
52     glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGBA);
53     glutInitWindowSize(800, 600);
54     glutCreateWindow("Script 3");
55
56     glutDisplayFunc(display);
57
58     glutMainLoop();
59
60     return 0;
61 }
```

Listing 3: Código-fonte adicional.